MBAi 460
Modern Software Architectures
Spring 2026

Project #02, part 01 (v1.0)

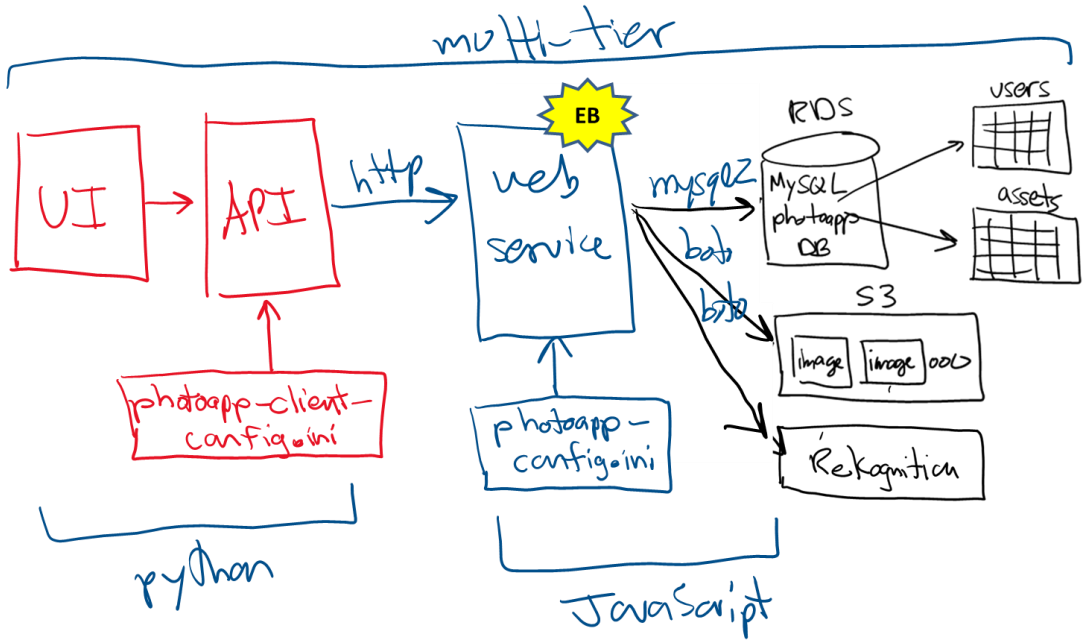
Assignment: Multi-tier PhotoApp with web service
Submission: via Gradescope
Policy: team-oriented, late work is accepted
Complete By: Monday May 4th @ 11:59pm CST

Late submissions: see syllabus for late policy… No submissions accepted after Thurs 05/07 @ 11:59pm

Pre-requisites: Designing and building web services

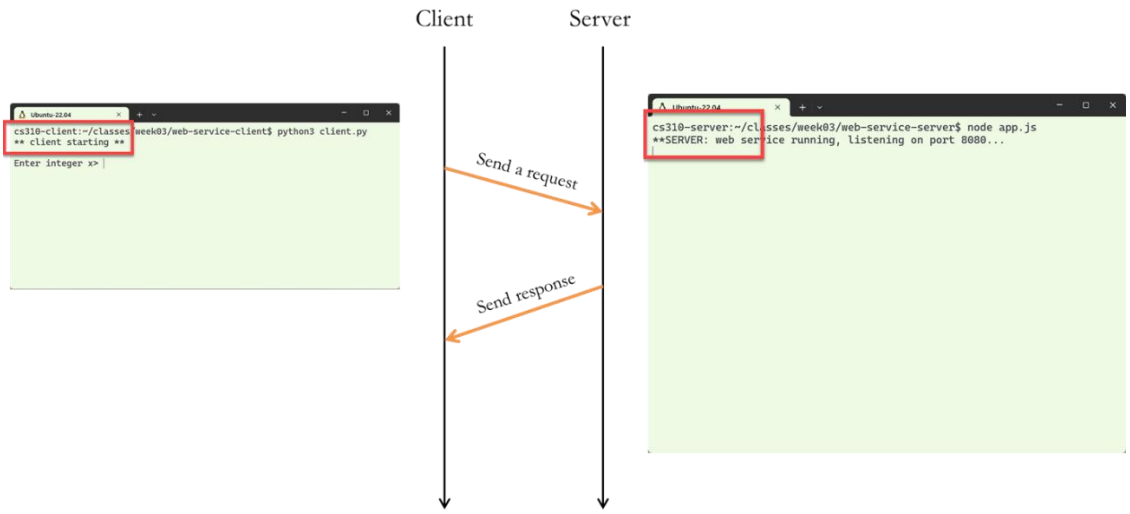
Overview

In project 02 we're rebuilding ***PhotoApp*** as a multi-tier application with a Node.js-based web service. The visual below provides a good overview of the components: (a) client-side UI and API functions in Python, (b) web service in JavaScript deployed via EB, and (c) server-side RDS, S3, and Rekognition services. Notice there are now 2 configuration files: "photoapp-client-config.ini" will contain the endpoint for the web service, while "photoapp-config.ini" remains unchanged but is now deployed as part of the web service. We recommend approaching the project in 3 steps: (1) build web service, (2) rewrite the API functions to interact with web service, and (3) deploy web service via Elastic Beanstalk using the scripts developed in Lab 03.

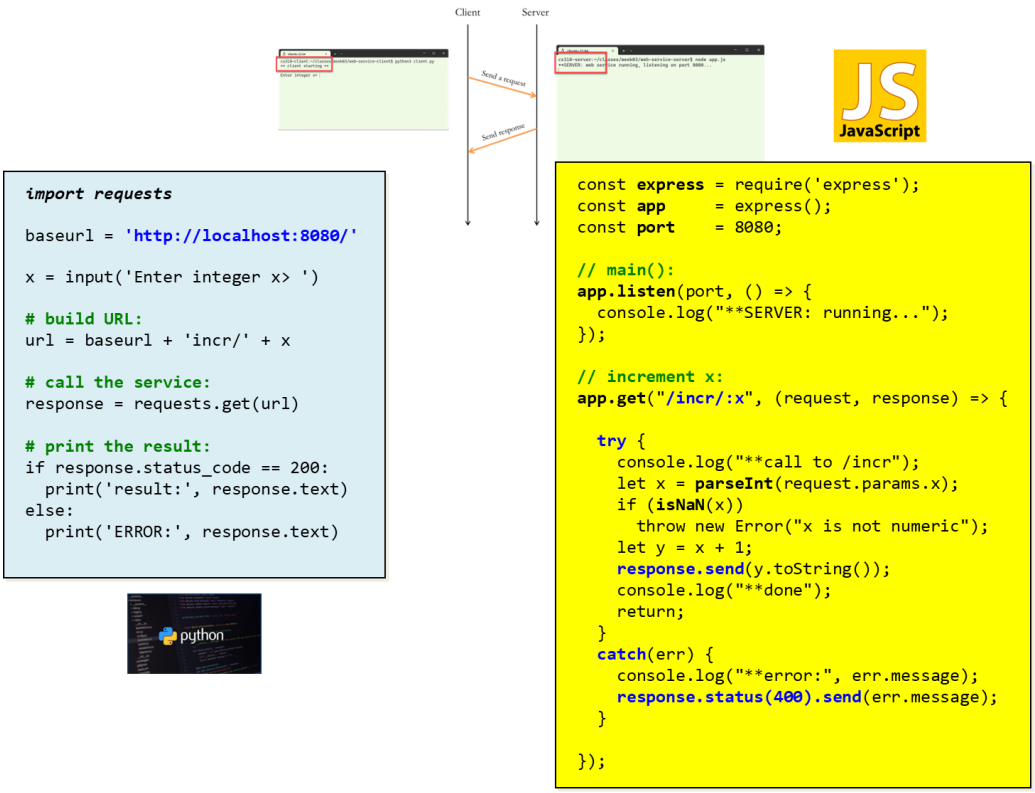


Before you get started...

Development of a multi-tier app is best done locally, with the LAST step being deployment to AWS. It's much easier to debug things locally vs. debugging apps running in the cloud. This implies you are going to have TWO docker images running on your local laptop: one for the client (Python) and one for the web service (Node.js):



If you are not using Docker, you'll need to install Node.js and Express locally on your laptop; installing Node.js is similar to installing Python, and well-documented (e.g. link). You'll also need some additional Node.js libraries such as *mysql2*, *boto*, and *p-retry*. These are typically captured in a "package.json" file (available in dropbox) which you can download and use when installing Node.js. As review, the following lecture slide summarizes the programming model of each component:



Working as part of a team? You can form new teams in project 02…

This is a non-trivial programming assignment, though completely within the realm of current AI technology. You can work independently or in a team (max size 4), up to you. We are not assuming the same teams as project 01, so If you plan to work to work in a team on project 02 please do the following:

1. *Login to canvas*
2. *Click on "People"*
3. *Click on "Groups"*
4. *Scroll down to the groups named "**Project 02 1**", "**Project 02 2**", etc.*
5. *Pick a group that has no members, and add yourself to the group. Have your teammates add themselves to the same group.*

That's it! Note that submissions will be collected via Gradescope, not Canvas. When you submit to Gradescope, only one team member needs to submit; every team member receives the same score.

Using Docker? Server-side setup…

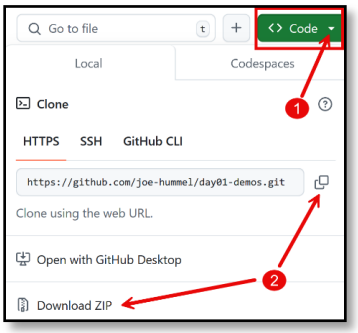
You already have the necessary client-side Docker image (or local install) setup from Project 01. If you plan to run the web service using Docker, download and build the provided "mbai460-server" image. Steps:

a. **Browse to** https://github.com/joe-hummel/mbai460-server

b. As shown in the screenshot on the right, **download** the provided files using "Download ZIP". [*If you are comfortable using git, feel free to clone the repo.*]

c. Once downloaded, extract the files from the ZIP file. After unzipping you should have a folder named "mbai460-server-main".

d. Using your local Mac OS / Windows / Linux, move the folder to the same location as the client-side image folder "mbai460-client-main". Remember that the Docker folder(s) need to reside in a location that is NOT automatically cloud sync'd.



After downloading, open the "readme.txt" and follow the instructions appropriate for your platform: Linux, Mac, or Windows. Build the image… Assuming the image was built successfully, run the image, cd into the test folder, and test via the supplied web service in "app.js" (the simple calculator from class):

```
hummel> ./docker/run
cs310-server:~$ cd test
cs310-server:~/test$ node app.js
**SERVER: web service running, listening on port 8080...
```

Now open a browser and browse to http://localhost:8080/incr/123 and if all is well, you should get 124 as the response.

Client-side starter files

An initial set of client-side AND servers-side files are provided to help you get started. These files are available in both the client and server docker images. In the **client** docker image, open the "projects" folder, and then open the sub-folder "project02". Open the "client" sub-folder & you should see the following files:

00no-labels.jpg	03rollcloud.jpg	photoapp-client-config.ini
01degu.jpg	04sailing.jpg	photoapp.py
02earth.jpg	client.py	tests.py

These are conceptually the same files are project 01, the major difference being that the API functions in "photoapp.py" are written to interact with the web service pointed to by "photoapp-client-config.ini":

photoapp-client-config.ini

```
[client]
webservice=http://localhost:8080
```

Three of the API functions have been written for you: ***initialize( )***, ***get_ping( )***, and ***get_users( )***. All the client-side API functions must behave as they did in project 01, hiding the fact that they are now communicating with a web service. One change was necessary to **get_ping( )**: it now returns an exception if S3 or MySQL is not reachable. Here's the code for *get_ping( )*, which is the template you should follow for all client-side API functions. Notice the ***retry*** decoration has been modified to retry ONLY if there's a connection or timeout error when trying to reach the web service. If an error occurs within the web service, we assume the web service has retried to resolve the error on their side (so we don't need to call the web service again):

```
##################################################################
#
# get_ping
#
# To "ping" a system is to see if it's up and running. This
# function pings the bucket and the database server to make
# sure they are up and running. Returns a tuple (M, N), where
#
#   M = # of items in the photoapp bucket
#   N = # of users in the photoapp.users table
#
# If an error occurs / a service is not accessible, M or N
# will be an error message. Hopefully the error messages will
# convey what is going on (e.g. no internet connection).
#
@retry(stop=stop_after_attempt(3),
        wait=wait_exponential(multiplier=1, min=2, max=30),
        retry=retry_if_exception_type((ConnectionError, Timeout)),
        reraise=True
      )
def get_ping():
  """
  Based on the configuration file, retrieves the # of items in the S3 bucket and
  the # of users in the photoapp.users table. Both values are returned as a tuple
  (M, N). If an error occurs, e.g. S3 or the database is unreachable, then an
```

```
  exception is raised.

  Parameters
  ----------
  N/A

  Returns
  -------
  the tuple (M, N) where M is the # of items in the S3 bucket and
  N is the # of users in the photoapp.users table. If an error
  occurs, e.g. S3 or the database is unreachable, then an exception
  is raised.
  """
  try:
    baseurl = WEB_SERVICE_URL
    url = baseurl + "/ping"

    response = requests.get(url)

    if response.status_code == 200:
      #
      # success
      #
      body = response.json()
      M = body['M']
      N = body['N']
      return (M, N)
    elif response.status_code == 500:
      #
      # failed:
      #
      body = response.json()
      msg = body['message']
      err_msg = f"status code {response.status_code}: {msg}"
      #
      # NOTE: this exception will not trigger retry mechanism,
      # since we reached the server and the server-side failed,
      # and we are assuming the server-side is also doing retries.
      #
      raise HTTPError(err_msg)
    else:
      #
      # something unexpected happened, and in this case we don't
      # have a JSON-based response, so let Python raise proper
      # HTTPError for us:
      #
      response.raise_for_status()

  except Exception as err:
    logging.error("get_ping():")
    logging.error(str(err))
    #
    # raise exception to trigger retry mechanism if appropriate:
    #
    raise
```

Most importantly, notice the function no longer interacts with MySQL or S3, but instead makes a ***get*** request

to the "***/ping***" web service function. The response is returned in JSON format, which is deserialized and the appropriate value(s) extracted and returned (or an exception is raised). Errors from the web service are raised as HTTPErrors on the client.

Take a moment to review the code for ***get_users( )***, which has to post-process the data returned from the web service… That's it for the client-side files.

Server-side starter files

Now let's work with the server-side files… If you haven't already, run the **server** docker image. Open the "projects" folder, and then open the sub-folder "project02". Open the "server" sub-folder & you should see the following files:

api_get_ping.js	app.js	helper.js
api_get_users.js	config.js	

As with the client, the server-side equivalent functions "**/ping**" and "**/users**" have been written in the files "api_get_ping.js" and "api_get_users.js" respectively. At the very end of the startup file "app.js", you'll see it loads these files and connects them with the appropriate HTTP verb *get*, *put*, *post*, *delete*:

```
//
// PhotoApp web service based on Node.js and Express. This file
// contains the main function that starts and listens on the
// configured network port. The remaining API functions are
// defined in separate JS files for easier development.
//
// Authors:
//  YOUR NAME
//  Prof. Joe Hummel (initial template)
//  Northwestern University
//

const express = require('express');
const app = express();
const config = require('./config.js');

// support larger image uploads/downloads:
app.use(express.json({ strict: false, limit: "50mb" }));


/**
 * main:
 *
 * @description startup code for web service, starts listening on port
 * @param none
 * @returns none
 */
var startTime;

app.listen(config.web_service_port, () => {
  startTime = Date.now();
```

```
  console.log(`**Web service running, listening on port ${config.web_service_port}...`);
  //
  // Configure AWS to use our config file:
  //
  process.env.AWS_SHARED_CREDENTIALS_FILE = config.photoapp_config_filename;
});


/**
 * get /
 *
 * @description handles request for what would be default page as if
 * we were a web server. Returns startus and how long the service has
 * been up and running (seconds).
 *
 * @param request
 * @param response
 * @returns {status: string, uptime_in_seconds: integer}
 */
app.get('/', (request, response) => {
  try {
    console.log("**Call to /...");

    let uptime = Math.round((Date.now() - startTime) / 1000);

    console.log("sending response...");

    response.json({
      "status": "running",
      "uptime_in_secs": uptime,
    });
  }
  catch(err) {
    console.log("ERROR:");
    console.log(err.message);

    //
    // if something goes wrong it's our fault, ==> use a
    // status code of 500 ==> server-side error:
    //
    response.status(500).json({
      "status": err.message,
      "uptime_in_secs": uptime,
    });
  }
});


//
// web service API functions, one per JS file:
//
//
// 1. app.get('/ping', async (request, response) => {...});
//
let get_ping_file = require('./api_get_ping.js');
app.get('/ping', get_ping_file.get_ping);
//
// 2. app.get('/users', async (request, response) => {...});
```

```
//
let get_users_file = require('./api_get_users.js');
app.get('/users', get_users_file.get_users);

//
// load and define more API functions here:
//
```

As you write additional server-side API functions, you'll need to modify "app.js" to load the underlying .js file and connect it with the appropriate HTTP verb.

Let's look at the code in "api_get_ping.js" for the "**/ping**" web service function. The highlights:

1. the asynchronous nature of the inner functions ***get_M( )*** and ***get_N( )***
2. the ***mysql2*** library used by *get_N( )* supports Promises, unlike the *mysql* library demonstrated in the lecture slides
3. the use of ***Promise.all( )*** to allow the inner functions to run concurrently
4. the use of Node.js's ***p-retry*** library to retry failed calls to *get_N( )* since the underlying ***mysql2*** library does not perform retries; this is not necessary with calls to *get_M( )* since it uses the underlying AWS ***boto*** library which supports retries. Here's the line of code using p-retry:

   ```
   let promise_mysql = await pRetry( () => get_N(), {retries: 2} );
   ```

   The first parameter to pRetry is a lambda expression that calls the function using retry logic.
5. sends a response in JSON format, using a dictionary-like object of the form {"message": string, "M": int, "N": int}
6. catches exceptions and sends a response with status code 500 (server error) in the same JSON format, where "message" carries the error message

```
//
// API function: get /ping
//
// Returns (M, N) where M = # of items in the S3 bucket and
// N = # of users in the database.
//
// Author:
//   Prof. Joe Hummel
//   Northwestern University
//

const mysql2 = require('mysql2/promise');
const { get_dbConn, get_bucket, get_bucket_name } = require('./helper.js');
const { ListObjectsV2Command } = require('@aws-sdk/client-s3');
//
// p_retry requires the use of a dynamic import:
// const pRetry = require('p-retry');
//
const pRetry = (...args) => import('p-retry').then(({default: pRetry}) =>
pRetry(...args));

/**
 * get_ping:
```

```
 *
 * @description returns (M, N) where M = # of items in the bucket and
 * N = # of users in the database. The format of the response is
 * {message: ..., M: ..., N: ...} where message is either "success" or
 * an error message (with status code 500). If message is an error
 * message, then M and N will be -1.
 *
 * @param none
 * @returns JSON {message: string, M: integer, N: integer}
 */
exports.get_ping = async (request, response) => {

  async function get_M()
  {
    //
    // call out to S3 and get the items in the bucket...
    //
    let bucket = get_bucket();

    let parameters = {
      Bucket: get_bucket_name(),
    };

    console.log("calling S3...");

    let command = new ListObjectsV2Command(parameters);
    let promise_s3 = bucket.send(command);

    return promise_s3;
  }

  async function get_N()
  {
    try {
      //
      // open connection to database and start query execution to
      // count # of users:
      //
      dbConn = await get_dbConn();

      let sql = `
                SELECT count(userid) AS num_users
                FROM users;
                `;

      console.log("executing SQL...");

      let promise_mysql = dbConn.execute(sql);

      return promise_mysql;
    }
    finally {
      //
      // close connection:
      //
      try { await dbConn.end(); } catch(err) { /*ignore*/ }
    }
  }
```

```
try {
  console.log("**Call to get /ping...");

  let promise_s3 = get_M();
  let promise_mysql = pRetry( () => get_N(), {retries: 2} );

  // wait for database and S3 to complete, if an error occurs
  // then an exception is thrown:
  let results = await Promise.all([promise_s3, promise_mysql]);

  // calls have completed, extract results from promises:
  console.log("operations completed...");

  //
  // S3 call is returning a list of the objects in the bucket,
  // so we return the length of the list:
  //
  let results_s3 = results[0];
  let contents = results_s3["Contents"];
  let M = contents.length;

  //
  // SQL is returning count of # of users, so extract from
  // result set:
  //
  let results_mysql = results[1];
  let [rows, _] = results_mysql;
  let row = rows[0];  // we only have one result back: num_users
  let N = row["num_users"];

  console.log(`M: ${M}`);
  console.log(`N: ${N}`);

  //
  // success:
  //
  console.log("sending response...");

  response.json({
    "message": "success",
    "M": M,
    "N": N,
  });

}
catch (err) {
  //
  // exception:
  //
  console.log("ERROR:");
  console.log(err.message);

  //
  // if an error occurs it's our fault, so we use status code
  // 500 => server-side error:
  //
  response.status(500).json({
```

```
      "message": err.message,
      "M": -1,
      "N": -1,
    });
  }

};
```

Initial testing

At this point the provided code is complete and should run without error. First, locate your "photoapp-config.ini" configuration file from Project 01, and copy this file into the "server" folder. Now open a Terminal / PowerShell window, and if you're using Docker then start the ***server*** image running. Now start the web service:

```
hummel> ./docker/run
cs310-server:~$ cd projects
cs310-server:~/projects$ cd project02
cs310-server:~/projects/project02$ cd server
cs310-server:~/projects/project02/server$ node app.js
**Web service running, listening on port 8080...
```

Now open another Terminal / PowerShell window, and if you're using Docker then start the client image running. Now run the provided client code:

```
hummel> ./docker/run
cs310-client:~$ cd projects
cs310-client:~/projects$ cd project02
cs310-client:~/projects/project02$ cd client
cs310-client:~/projects/project02/client$ python3 client.py

**starting**

initializing:
True

**get_ping:
M: 104
N: 3

**get_users:
(80001, 'p_sarkar', 'Pooja', 'Sarkar')
(80002, 'e_ricci', 'Emanuele', 'Ricci')
(80003, 'l_chen', 'Li', 'Chen')

**done**
```

Your value for M should be much smaller, but otherwise the output should match would you see above. If you flip back to the server window, you'll see the output from the console.log( ) calls in the server code:

```
hummel> ./docker/run
cs310-server:~$ cd projects
cs310-server:~/projects$ cd project02
cs310-server:~/projects/project02$ cd server
cs310-server:~/projects/project02/server$ node app.js
**Web service running, listening on port 8080...
**Call to get /ping...
calling S3...
executing SQL...
operations completed...
M: 104
N: 3
sending response...
**Call to get /users...
executing SQL...
done, retrieved 3 rows
success, sending response...
```

You can stop the web service at any time via ctrl-C (on a Mac you also use the Ctrl key, not Command).

Web service function: get /images

Okay, it's time to start implementing the web service functions. As you implement each function in the web service, update the corresponding client-side API function to call the web service, and then test by calling the API function from "client.py" or "tests.py" (or both). As shown in the provided client-side and server-side code, all client-side API functions are required to use retry decorations to retry at most 3 times. Likewise, all web service functions are required to use retry logic in all MySQL-based calls, retrying at most 3 times.

Let's start by implementing the "**/images**" function, which is very similar to the provided "**/users**" function. The header comment is shown below (you should copy-paste these header comments into your .js files):

1. *copy "api_get_users.js" and name the file "api_get_images.js"*
2. *update "app.js" to load the file and link it with the GET verb (see code @ end of "app.js" for examples); in this case "app.js" is calling the function* **get_images** *defined within "api_get_images.js"*
3. *rewrite "api_get_images.js" to query the database and respond appropriately. Note that the optional userid is passed as a query parameter, not a URL parameter (see lecture notes for week 03). When the optional parameter is present and you are building the SQL string, note that the* ***mysql2*** *library in Node.js uses* ***?*** *to denote placeholders in SQL strings, not %s that was used in Python's pymsql library.*
4. *within "api_get_images.js" be sure to change all references to "get_users" to "get_images", in particular the function name and the output from console.log( )*

```
/**
 * get_images:
 *
 * @description returns all the images in the database as a JSON object
 * {message: ..., data: ...} where message is either "success" or an
 * error message (with status code 500). If successful, the data is a list of
 * dictionary-like objects of the form {"assetid": int, "userid": int,
 * "localname": string, "bucketkey": string}, in order by assetid. If an error
 * occurs then the list is empty []. If a userid is given, then just the images
 * with that userid are returned; validity of the userid is not checked,
```

```
 * which implies that an empty list is returned if the userid is invalid.
 *
 * @param userid (optional query parameter) filters the returned images for just
this userid
 * @returns JSON {message: string, data: [object, object, ...]}
 */
```

Once the web service is implemented, the quickest way to test a GET function is to run the web service, open a browser, and browse to **http://localhost:8080/images** . You can test the handling of the optional parameter by adding a query parameter, e.g. **http://localhost:8080/images?userid=80001** . When the web service function is working, update the client-side API function to issue a GET request to "/images" (with retry decoration). Then modify "client.py" or "tests.py" (or both) to call the API function and in turn the web service. Finally, start the the web service (if not already running), switch to your client-side Terminal / PowerShell window, and run the client / tests.

Web service function: post /image

The next web service function is POST "**/image/:userid**", which uploads an image to S3. We use the verb POST because each time this function is called we upload the image, regardless of whether the *local_filename* parameter is a duplicate or not. The .js file should be called "api_post_image.js", and define a function named ***post_image( )***. Then modify "app.js" to load this file, and link the *POST* verb to the *post_image* function. Here's the header comment for *post_image* (which you should copy-paste into your .js file):

```
/**
 * post_image:
 *
 * @description uploads an image with a unique name to S3, allowing the
 * same local file to be uploaded multiple times if desired. A record
 * of this image is inserted into the database, and upon success a JSON
 * object of the form {message: ..., assetid: ...} is sent where message is
 * "success" and assetid is the unique id for this image in the database.
 * The image is also analyzed by the Rekognition AI service to label
 * objects within the image; the results of this analysis are also saved
 * in the database (and can be retrieved later via GET /image_labels).
 * If an error occurs, a status code of 500 is sent where the JSON object's
 * message is the error message and assetid is -1. An invalid userid is
 * considered a client-side error, resulting in a status code 400 with
 * a message "no such userid" and an assetid of -1.
 *
 * @param userid (required URL parameter) for whom we are uploading this image
 * @param request body {local_filename: string, data: base64-encoded string}
 * @returns JSON {message: string, assetid: int}
 */
```

This function is unlike the web service functions we have seen so far. In this case the web service is running on a different computer in the cloud, and doesn't have access to the client's computer --- so passing a local filename from client to web service will not allow the web service to upload the image to S3. Instead, the client needs to pass the ***CONTENTS*** of the image to the web service, i.e. the 0's and 1's that constitute the image. How is this done?

In the world of web servers, data is passed from a client to the server in the ***body*** of the request ***message***. Since we are using JSON --- a string-based format --- the binary image file of 0's and 1's must be converted into a ***base64-encoded string*** and sent to the web service. The web service receives the request, converts the base64-encoded string back into it's original binary form, and then uploads to S3.

With that background, let's start implementing the contents of "api_post_image.js". First, add the following near the top along with the other require( ) statements:

```
const { PutObjectCommand } = require("@aws-sdk/client-s3");
const { DetectLabelsCommand } = require("@aws-sdk/client-rekognition");
const uuid = require("uuid");
```

Like we did in project 01, think about the order of operations for posting an image… You'll need to use transactions as you are modifying the database… Also think about the use of inner functions, as you will need retry logic for MySQL-based calls… When you need to create a unique bucket key, you can use the same UUID-based approach here in JavaScript:

```
// generate a unique string for the bucketkey:
let unique_str = uuid.v4();
```

When you're ready to upload to S3, extract the local filename and image contents from the body of the request as follows:

```
let local_filename = request.body.local_filename;
let image_base64   = request.body.data;
let image_bytes    = Buffer.from(image_base64, 'base64');
```

Buffer is built-in Node.js functionality that supports various forms of decoding and encoding. To upload the decoded image bytes to S3, we send a ***PutObjectCommand***:

```
let parameters = {
  Bucket: get_bucket_name(),
  Key: bucketkey,
  Body: image_bytes,
};

let command = new PutObjectCommand(parameters);
let promise_s3 = bucket.send(command);
```

This would be a good time to switch to the client-side and test the basic upload functionality of the web service. The client-side API function sends a POST request to the web service, sending both the userid (as part of the URL) along with the necessary data --- local filename and image contents --- in the body of the request:

```
#
# first, open the file and read in the bytes
#
infile = open(local_filename, "rb")
image_bytes = infile.read()
infile.close()

#
# now encode as base64 string, build msg to embed in the request,
# and post request to the web service in JSON format:
#
encoded_bytes = base64.b64encode(image_bytes)  # returns bytes object
image_str     = encoded_bytes.decode()         # decode to obtain str

data = {"local_filename": local_filename, "data": image_str}

response = requests.post(url, json=data)  # post data in body of request:
```

At this point you should be able to test your work by making a call to the client-side API function, which in turn calls the web service to upload the image to S3. You can then check the database and S3 to confirm they have been properly updated. You should also download the image from S3 and open it locally to confirm it was uploaded correctly. [***Hint****: we are leaving some web service details for you to figure out, e.g. and how to use transactions with* ***mysql2****, and the exact format of responses from S3 and Rekognition. When in doubt, in the web service functions use* ***console.log( )*** *to output a response and discover the format. Likewise, on the client-side use Python's* ***print( )*** *to output a response and discover its format.*]

Once you can successfully upload images via POST "**/image**", be sure to add the necessary retry decoration to the client-side API function if you haven't already. Now, back in the web service, add support to call Rekognition to detect labels and store in the database (like we did in project 01). Back in the web service file "api_post_image.js", you can call Rekognition as follows:

```
let parameters = {
  Image: {
    S3Object: {
      Bucket: get_bucket_name(),
      Name: bucketkey,
    },
  },
  MaxLabels: 100,
  MinConfidence: 80.0,
};

let command = new DetectLabelsCommand(parameters);
let promise_rekognition = rekognition.send(command);
```

Run and test with your existing client-side code, and confirm the database is updated appropriately with the correct labels. Be sure to test any edge cases, e.g. an invalid userid or local filename --- the userid is validated by the web service, but the local filename must be checked by the client-side API function. There's really no way for the web service to detect an invalid local filename or image contents, so the parameters in the body of the request are accepted as given.

Web service function: get /image

The GET "**/image/:assetid**" web service function downloads an image from S3. In this case the contents of the image are carried in the *response* as a base64-encoded string in JSON format. The client then deserializes the JSON, decodes the base64 string, and stores the resulting binary image bytes in a file. The .js file should be called "api_get_image.js", and define a function named ***get_image( )***. Then modify "app.js" to load this file, and link the *GET* verb to the *get_image* function. Here's the header comment for the web service function:

```
/**
 * get_image:
 *
 * @description downloads an image denoted by the given asset id. If
 * successful a JSON object of the form {message: string, userid: int,
 * local_filename: string, data: base64-encoded string} is sent where
 * message is "success" and the remaining values are set based on the
 * image downloaded; data is image contents as base64-encoded string.
 * If an error occurs, a status code of 500 is sent where JSON object's
 * message is the error message and userid is -1; the other values are
 * undefined. An invalid assetid is considered a client-side error,
 * resulting in a status code 400 with a message "no such assetid"
 * and a userid of -1; the other values are undefined.
 *
 * @param assetid (required URL parameter) of image to download
 * @returns JSON {message: string, userid: int, local_filename: string,
data: base64-encoded string}
 */
```

Create a web service file "api_get_image.js" (perhaps by copying "api_post_image.js"?), defining a function named ***get_image( )***. Then modify "app.js" to load this file, and link the *GET* verb to the *get_image* function. At the top of the file add the following require( ) statement (you'll need others as well):

```
const { GetObjectCommand } = require("@aws-sdk/client-s3");
```

You'll have to validate the assetid, and then if valid, download the image from S3 and pass along the image contents to the client as a base64-encoded string. Interestingly the ***transformToString( )*** in Node.js is asynchronous to handle encoding of different possible streams of input:

```
//
// download image from S3:
//
let parameters = {
  Bucket: get_bucket_name(),
  Key: bucketkey,
};

let command = new GetObjectCommand(parameters);
let promise_s3 = bucket.send(command);
```

```
let results_s3 = ???  // wait for promise to resolve:

//
// we have the image in raw bytes, now transform it into
// base-64 string so we can transmit to client:
//
let promise_image_str = results_s3.Body.transformToString("base64");

let image_str = ???  // wait for promise to resolve:
```

The web service then builds the appropriate JSON message and responds to the client. On the client, the API function issues a GET request to the web service, receives the response, and if the call was successful, decodes and stores the image contents:

```
#
# call web service to download image:
#
response = requests.get(url)

#
# if successful, extract the image contents, decode, and store
# as a binary file:
#
if response.status_code == 200:
  body = response.json()
  image_base64 = body["???"]
  image_bytes  =  base64.b64decode(image_base64)
  #
  # TODO: write image_bytes to a binary file
  #
  return local_filename
elif response.status_code in [400,500]:
  .
  . # TODO: raise exception based on JSON code and msg from web service
  .
else:
  .
  . # TODO: raise exception using response.raise_for_status()
  .
```

Build, run and test. Be sure the client-side API function has the appropriate retry decoration to retry at most 3 times. Likewise, the web service function should have the appropriate retry logic for any MySQL-based calls.

Web service function: get /image_labels

Three web service and API functions left… The GET "**/image_labels/:assetid**" web service function retrieves the Rekognition-generated labels from the database for a given image. The .js file should be called "api_get_image_labels.js", and define a function named ***get_image_labels( )***. Then modify "app.js" to load this file, and link the *GET* verb to the *get_image_labels* function. Here's the header comment:

```
/**
 * get_image_labels
 *
 * @description when an image is uploaded to S3, the Rekognition
 * AI service is automatically called to label objects in the image.
 * Given the image assetid, this function retrieves those labels.
 * If successful the labels are returned as a JSON object of the
 * form {message: ..., data: ...} where message is "success" and
 * data is a list of dictionary-like objects of the form
 * {"label": string, "confidence": int}, ordered by label. If
 * an error occurs, status code of 500 is sent where JSON object's
 * message is the error message and the list is empty []. An
 * invalid assetid is considered a client-side error, resulting
 * in status code 400 with a message "no such assetid" and an empty
 * list [].
 *
 * @param assetid (required URL parameter) of image to retrieve labels for
 * @returns JSON {message: string, data: [object, object, ...]}
 */
```

Once the web service function is in place with the appropriate retry logic, update "app.js" to link in the new function. Then update the client-side API function to issue a GET request to "**/image_labels**", adding the required retry decoration to retry at most 3 times. Then modify "client.py" or "tests.py" (or both) to call the API function and in turn the web service, run and test.

Web service function: get /images_with_label

The GET "**/images_with_label/:label**" web service function searches the Rekognition-generated labels in the database for those containing the given label. The .js file should be called "api_get_images_with_label.js", and define a function named ***get_images_with_label( )***. Then modify "app.js" to load this file, and link the *GET* verb to the *get_images_with_labels* function. Here's the header comment:

```
/**
 * get_images_with_label
 *
 * @description when an image is uploaded to S3, the Rekognition
 * AI service is automatically called to label objects in the image.
 * These labels are then stored in the database for retrieval / search.
 * Given a label (partial such as 'boat' or complete 'sailboat'), this
 * function performs a case-insensitive search for all images with
 * this label. If successful the labels are returned as a JSON object
 * of the form {message: ..., data: ...} where message is "success" and
 * data is a list of dictionary-like objects of the form {"assetid": int,
 * "label": string, "confidence": int}, ordered by assetid and then label.
 * If an error occurs, status code of 500 is sent where JSON object's
 * message is the error message and the list is empty.
```

```
 *
 * @param label (required URL parameter) to search for, can be a partial word (e.g.
boat)
 * @returns JSON {message: string, data: [object, object, ...]}
 */
```

Once the web service function is in place with the appropriate retry logic, update "app.js" to link in the new function. Then update the client-side API function to issue a GET request to "**/images_with_label**", adding the required retry decoration to retry at most 3 times. Then modify "client.py" or "tests.py" (or both) to call the API function and in turn the web service, run and test.

Web service function: delete /images

Last function! The DELETE "**/images**" web service function deletes all images and associated labels from the database and S3. The .js file should be called "api_delete_images.js", and define a function named ***delete_images( )***. Then modify "app.js" to load this file, and link the *DELETE* verb to the *delete_images* function. Here's the header comment:

```
/**
 * delete_images:
 *
 * @description deletes all images and associated labels from the
 * database and S3. If successful, returns a JSON object of the
 * form {message: string} where the message is "success". If an
 * error occurs, message will carry the error message. The images
 * are not deleted from S3 unless the database is successfully
 * cleared; if an error occurs either (a) there are no changes or
 * (b) the database is cleared but there may be one or more images
 * remaining in S3 (which has no negative effect since they have
 * unique names).
 *
 * @param none
 * @returns JSON {message: string}
 */
```

When updating the database, be sure to use a transaction. And note that with Node.js's ***mysql2*** library, you need to use the ***query( )*** function --- not the *execute( )* function --- to execute the multi-statement query given in project 01 for truncating the tables and resetting the auto-increment on the *assets* table:

```
let sql = `
          SET foreign_key_checks = 0;
          TRUNCATE TABLE labels;
          TRUNCATE TABLE assets;
          SET foreign_key_checks = 1;
          ALTER TABLE assets AUTO_INCREMENT = 1001;
          `;
```

To delete objects from S3 you'll need to create a ***DeleteObjectsCommand*** object and send to S3. First, add the following require( ) statement near the top of your .js file (along with the others you'll need):

```
const { DeleteObjectsCommand } = require("@aws-sdk/client-s3");
```

When you're ready to delete the images from S3, send the command to S3 as follows:

```
//
// delete images from S3:
//
let objects_to_delete = ?;  // an array of dictionary-like objects {Key: string}

let parameters = {
  Bucket: get_bucket_name(),
  Delete: {Objects: objects_to_delete},
};

let command = new DeleteObjectsCommand(parameters);
let promise_s3 = bucket.send(command);
```

Be sure to wait for the promise to resolve before responding to the client with success or failure! Modify the client-side API function to call the web service, and then modify client / tests to call the API. Run and test!

Deployment to AWS Elastic Beanstalk

At this point you should have a working Node.js-based web service, with an updated set of client-side API functions and testing code. Congratulations, you have built a realistic, multi-tier cloud application!

The last step is to deploy your web service to the cloud, using the scripts we will develop in Lab 03. But this handout is already 20+ pages long, so we're going to leave the details of EB deployment to Lab 03 and a separate "part 02" handout.

Grading and Electronic Submission

Submissions will be collected on Gradescope. There will be TWO submission sites, one for the web service and one for the client-side. For the web service, if you are running Docker **server** image you'll submit your server-side files as follows from your "server" sub-folder:

```
/gradescope/gs submit 1288073 8052758 *.js *.ini
```

If you are working outside Docker, you can drag-drop your .js and .ini files to "Project 02 – web service" on

Gradescope. The goal is an autograder score of 60/60; you have unlimited submissions.

Once you have passed the Gradescope tests for your web service, you'll submit your client-side API file "photoapp.py" AND your web service files to a different Gradescope site. The simplest approach is to copy your "photoapp.py" file --- which is in your CLIENT docker image --- over to your SERVER docker image's "server" folder. Then run this command to submit:

```
/gradescope/gs submit 1288073 8052765 *.js *.ini photoapp.py
```

If you are working outside Docker, you can drag-drop your .js, .py and .ini files to "Project 02 – client API" on Gradescope. The goal is an autograder score of 30/30; you have unlimited submissions.

By default, Gradescope records the score of your **LAST SUBMISSION** (not your highest submission score). If you want us to grade a different submission, activate the appropriate submission via your "Submission History". This must be done before the due date.

Pause your RDS server?

Once you have submitted to Gradescope and completed this part of the assignment, this is a good time to select your database instance and under "Actions", select "Stop temporarily" --- this will pause the server, free the assigned VM instance, and reduce the odds of you being charged (AWS has started charging for public IP addresses). However, you'll need to manually restart when you want to resume working on part 02. Recall you can start and stop your RDS server using the AWS CLI, where the DB_INSTANCE_NAME is something like *mysql-nu-cs310*:

```
aws rds stop-db-instance --db-instance-identifier DB_INSTANCE_NAME

aws rds start-db-instance --db-instance-identifier DB_INSTANCE_NAME
```